

GUÍA DE REPASO COMPLETA

Módulo 1 · Unidad 1 · Clase 4

Pandas: Ingesta, Limpieza y Análisis de Datos

¿Para quién es esta guía?

Esta guía cubre el ciclo completo de trabajo con Pandas: cargar datos desde CSV, Excel, JSON y SQL; detectar y tratar valores faltantes con `isna` y `fillna`; resumir y segmentar con `groupby`; combinar tablas con `merge`; y apilar DataFrames con `concat`. Cada operación se explica antes del código y cada resultado se interpreta en términos técnicos y de negocio.

Pandas es la librería más usada para análisis de datos en Python. Si NumPy es la base matemática, Pandas es la herramienta de trabajo diario del científico de datos. El 80% de las tareas de preparación de datos (Fase 3 de CRISP-DM) se resuelven con Pandas.

SECCIÓN 0 | Mapa Conceptual de la Clase

Los seis bloques de esta clase y su posición en el flujo de trabajo:

Bloque	Función en el flujo de datos
Ingesta (<code>read_csv</code> , <code>read_excel</code> , <code>read_json</code> , <code>read_sql</code>)	Carga datos desde su fuente original al DataFrame. El punto de entrada de todo análisis.
Exploración inicial (<code>.shape</code> , <code>.dtypes</code> , <code>.head</code> , <code>.info</code>)	Diagnóstico inicial: ¿cuántos datos? ¿qué tipos? ¿hay nulos evidentes?
Calidad de datos (<code>isna</code> , <code>fillna</code> , <code>dropna</code>)	Detecta y trata valores faltantes. Sin este paso, los análisis y modelos producen resultados incorrectos.
Columnas derivadas	Crea nuevas variables a partir de las existentes. Transforma datos crudos en métricas de negocio.
<code>groupby</code>	Segmenta y agrega. Responde preguntas como '¿cuál es el promedio por categoría?'
<code>merge</code> y <code>concat</code>	Combina múltiples fuentes de datos. La integración es donde los datos sueltos se convierten en análisis completo.

La conexión con CRISP-DM

Todo el contenido de esta clase corresponde a la Fase 2 (Comprensión de datos) y la Fase 3 (Preparación de datos) del ciclo CRISP-DM. La Fase 3 ocupa entre el 60% y el 80% del tiempo de un proyecto real. Dominar Pandas es, en la práctica, dominar la fase más

costosa de la ciencia de datos.

SECCIÓN 1 | ¿Qué es Pandas y cuál es su estructura central?

1.1 El DataFrame: la estructura de datos de Pandas

La estructura central de Pandas es el **DataFrame**: una tabla bidimensional con filas y columnas etiquetadas. Puedes pensar en él como una hoja de cálculo que vive en Python y sobre la que puedes aplicar operaciones matemáticas, filtros y transformaciones con una sola línea de código.

Pandas se construye sobre NumPy: cada columna de un DataFrame es internamente un array de NumPy. Por eso las operaciones vectorizadas de NumPy funcionan directamente sobre columnas de Pandas.

Concepto	Descripción	Analogía en Excel
DataFrame	Tabla 2D con filas y columnas etiquetadas. La estructura principal de Pandas.	La hoja de cálculo completa
Series	Una sola columna (o fila) del DataFrame. Array 1D con índice.	Una columna de la hoja
Índice (index)	Etiqueta de cada fila. Por defecto es 0, 1, 2... pero puede ser cualquier valor.	El número de fila en Excel
dtype	Tipo de dato de cada columna: int64, float64, object (texto), bool, datetime64.	El formato de celda en Excel

SECCIÓN 2 | Caso Desarrollado: EPM y el Análisis de Consumo Eléctrico

Contexto verificado: EPM

EPM (Empresas Públicas de Medellín E.S.P.) es la empresa de servicios públicos más grande de Colombia. Fue creada el 6 de agosto de 1955 mediante el Acuerdo #58 del Consejo Administrativo de Medellín, que fusionó en una sola entidad autónoma cuatro empresas entonces independientes: Energía, Acueducto, Alcantarillado y Teléfonos. Su único propietario es el Distrito de Medellín. Presta servicios de energía eléctrica, gas por

red, agua y saneamiento. En la última década ha transferido al Distrito de Medellín más de 16 billones de pesos para inversión social. Atiende a más de 7 millones de personas como empresa matriz.

Caso de uso: el equipo de analítica de EPM integra datos de consumo eléctrico (CSV), presupuesto de mantenimiento (Excel), tarifas por estrato (JSON) e incidentes de red (base de datos SQL) para generar un reporte consolidado de operaciones. Tú eres el analista a cargo de la integración y limpieza de datos.

Paso 1: Cargar datos desde CSV — `read_csv`

`read_csv()` es la función más usada de Pandas. Lee un archivo CSV y lo convierte directamente en un DataFrame. El CSV puede estar en disco, en una URL o generado en memoria.

```
import pandas as pd
import numpy as np

# Datos de consumo eléctrico por municipio — dos meses
# En producción, esto sería: pd.read_csv('ruta/al/archivo.csv')
# Aquí lo creamos desde un string para que el ejemplo sea reproducible
from io import StringIO

csv_data = '''municipio,mes,consumo_kwh,num_usuarios,tarifa_kwh
Medellín,2024-01,145230000,892000,650
Bello,2024-01,28450000,178000,640
Envigado,2024-01,19870000,124000,655
Itagüí,2024-01,22100000,138000,645
Medellín,2024-03,141890000,898000,660
Bello,2024-03,27890000,180000,650
Envigado,2024-03,19340000,126000,665
Itagüí,2024-03,21780000,140000,655'''

df_consumo = pd.read_csv(StringIO(csv_data))

# Exploración inicial — siempre los primeros pasos
print('Shape:', df_consumo.shape)
print(df_consumo.dtypes)
print(df_consumo.head(4))
```

```
Shape: (8, 5)
municipio      object
mes            object
consumo_kwh     int64
```

```
num_usuarios    int64
tarifa_kwh      int64
dtype: object
```

```
   municipio    mes  consumo_kwh  num_usuarios  tarifa_kwh
0  Medellín  2024-01    145230000         892000         650
1    Bello    2024-01    28450000         178000         640
2  Envigado  2024-01    19870000         124000         655
3   Itagüí    2024-01    22100000         138000         645
```

Interpretación: Paso 1

Interpretación técnica: shape (8, 5) indica 8 filas y 5 columnas. dtypes muestra que municipio y mes son object (texto), el resto son int64. La columna mes se cargó como texto, no como fecha: en un proyecto real se convertiría con `pd.to_datetime(df['mes'])` para poder hacer operaciones de tiempo.

Interpretación de negocio: Medellín domina el consumo (145M kWh en enero vs 28M de Bello). Tenemos datos de enero y marzo para 4 municipios del área metropolitana. La tarifa subió entre enero (650) y marzo (660) en Medellín, lo cual es información relevante para el cálculo de ingresos.

Paso 2: Cargar datos desde JSON — `read_json`

`read_json()` carga datos desde un archivo JSON. El JSON puede tener estructura de lista de objetos (cada objeto es una fila), que es el formato más común al consumir APIs REST.

```
import json

# Tarifas diferenciadas por estrato socioeconómico
# (EPM aplica subsidios a estratos 1, 2 y 3 y contribuciones a 5 y 6)
json_data = [

    {'municipio': 'Medellín', 'estrato': 1, 'subsidio_pct': 60, 'tarifa_final': 260},

    {'municipio': 'Medellín', 'estrato': 2, 'subsidio_pct': 50, 'tarifa_final': 325},

    {'municipio': 'Medellín', 'estrato': 3, 'subsidio_pct': 15, 'tarifa_final': 552},
    {'municipio': 'Medellín', 'estrato': 4, 'subsidio_pct':
0, 'tarifa_final': 650},
    {'municipio': 'Medellín', 'estrato': 5, 'subsidio_pct':
0, 'tarifa_final': 715},
    {'municipio': 'Medellín', 'estrato': 6, 'subsidio_pct':
0, 'tarifa_final': 780},
```

```

    {'municipio': 'Bello',
     'estrato': 1, 'subsidio_pct': 60, 'tarifa_final': 256},
    {'municipio': 'Bello',
     'estrato': 2, 'subsidio_pct': 50, 'tarifa_final': 320},
    {'municipio': 'Bello',
     'estrato': 3, 'subsidio_pct': 15, 'tarifa_final': 544},
    {'municipio': 'Bello', 'estrato': 4, 'subsidio_pct':
0, 'tarifa_final': 640},
]

df_json = pd.DataFrame(json_data)    # equivalente a pd.read_json()
print('Shape:', df_json.shape)
print(df_json)

```

```

Shape: (10, 4)
   municipio  estrato  subsidio_pct  tarifa_final
0  Medellín      1           60           260
1  Medellín      2           50           325
2  Medellín      3           15           552
3  Medellín      4            0           650
4  Medellín      5            0           715
5  Medellín      6            0           780
6    Bello       1           60           256
7    Bello       2           50           320
8    Bello       3           15           544
9    Bello       4            0           640

```

Interpretación: Paso 2

Interpretación técnica: `pd.DataFrame(lista_de_diccionarios)` convierte una estructura JSON estándar en DataFrame directamente. En producción, si los datos vienen de una API, se haría `requests.get(url).json() → pd.DataFrame(response)`.

Interpretación de negocio: EPM aplica subsidios del 60% al estrato 1 (tarifa final 260 COP/kWh vs tarifa base 650) y contribuciones extras a estratos 5 y 6 (780 COP/kWh). Este sistema de subsidios cruzados es regulado por la Ley 142 de 1994 (Ley de servicios públicos domiciliarios de Colombia).

Paso 3: Cargar datos desde SQL — `read_sql`

`pd.read_sql()` ejecuta una consulta SQL en una base de datos y devuelve el resultado directamente como DataFrame. Es la puerta de entrada entre el mundo SQL (Clase 3) y el mundo Pandas (Clase 4). La conexión puede ser SQLite, PostgreSQL, MySQL u otros.

```
import sqlite3

# Crear base de datos SQLite con datos de incidentes de red
conn = sqlite3.connect(':memory:')
df_consumo.to_sql('consumos', conn, index=False, if_exists='replace')

# Leer con una consulta SQL — aquí aplica todo lo aprendido en Clase 3
query = '''
    SELECT municipio,
           SUM(consumo_kwh) AS consumo_total,
           ROUND(AVG(tarifa_kwh), 0) AS tarifa_promedio
    FROM    consumos
    GROUP BY municipio
    ORDER BY consumo_total DESC
'''
df_sql = pd.read_sql(query, conn)
print(df_sql)
conn.close()
```

	municipio	consumo_total	tarifa_promedio
0	Medellín	287120000	655.0
1	Bello	56340000	645.0
2	Itagüí	43880000	650.0
3	Envigado	39210000	660.0

Interpretación: Paso 3

Interpretación técnica: `to_sql()` escribe el DataFrame en la base de datos (inverso de `read_sql()`). El parámetro `if_exists='replace'` sobrescribe la tabla si ya existe. `read_sql` devuelve un DataFrame limpio, listo para continuar el análisis en Python.

Interpretación de negocio: Medellín consume más del 67% del total combinado (287M de 426M kWh). Esta concentración es esperable dado que Medellín tiene 6-7 veces más usuarios que los otros municipios. `read_sql` permite aprovechar la potencia de SQL para la agregación y traer a Pandas solo el resumen, no los datos crudos (importante para eficiencia con millones de filas).

Paso 4: Detectar y tratar valores faltantes — `isna` y `fillna`

En datos reales, los valores faltantes (nulos) son inevitables: sensores desconectados, errores de captura, datos no disponibles. Pandas representa los valores faltantes como NaN (Not a Number). Antes de cualquier análisis o modelo, debes tener una estrategia explícita para cada columna con nulos.

```
# DataFrame de incidentes de red con valores faltantes intencionales
incidentes = {
    'id_incidente': [1, 2, 3, 4, 5, 6, 7, 8 ],
    'municipio': ['Medellín', 'Bello', 'Medellín', 'Envigado', None, 'Bello', 'Itagüí', 'Medellín'],
    'duracion_horas': [2.5, 1.0, None, 4.0, 3.5, None, 2.0, 1.5 ],
    'usuarios_afectados': [12000, 3400, 8900, None, 5600, 2100, None, 7800 ],
    'tipo': ['Falla red', 'Mantenimiento', 'Falla red', 'Tormenta', 'Falla red', 'Mantenimiento', 'Tormenta', None]
}
df_inc = pd.DataFrame(incidentes)

# Diagnóstico de nulos
print('Nulos por columna:')
print(df_inc.isna().sum())
print(f'\nTotal nulos: {df_inc.isna().sum().sum()}')
print(f'% de celdas con nulos: {100*df_inc.isna().sum().sum()/(df_inc.shape[0]*df_inc.shape[1]):.1f}%')
```

```
Nulos por columna:
id_incidente      0
municipio          1
duracion_horas    2
usuarios_afectados 2
tipo              1
dtype: int64

Total nulos: 6
% de celdas con nulos: 15.0%
```

```
# Estrategia de imputación por columna:
# - duracion_horas: imputar con la media (variable continua, sin outliers extremos)
# - usuarios_afectados: imputar con la mediana (más robusta ante el valor extremo de 12000)
# - municipio: imputar con 'Desconocido' (no podemos inferir la zona)
# - tipo: imputar con 'Sin clasificar' (categoría de respaldo)

df_limpio = df_inc.copy() # SIEMPRE trabajar sobre una copia

media_duracion = df_inc['duracion_horas'].mean()
mediana_usuarios = df_inc['usuarios_afectados'].median()

print(f'Media duración: {media_duracion:.4f} horas')
```

```

print(f'Mediana usuarios afectados: {mediana_usuarios:.0f}')

df_limpio['duracion_horas'] =
df_limpio['duracion_horas'].fillna(media_duracion)
df_limpio['usuarios_afectados'] =
df_limpio['usuarios_afectados'].fillna(mediana_usuarios)
df_limpio['municipio'] =
df_limpio['municipio'].fillna('Desconocido')
df_limpio['tipo'] = df_limpio['tipo'].fillna('Sin
clasificar')

print('\nNulos después de fillna:')
print(df_limpio.isna().sum())

```

```

Media duración: 2.4167 horas
Mediana usuarios afectados: 6700

Nulos después de fillna:
id_incidente      0
municipio          0
duracion_horas    0
usuarios_afectados 0
tipo              0
dtype: int64

```

Interpretación: Paso 4

Interpretación técnica: `df_inc.copy()` es crítico: modificar el DataFrame original puede causar bugs difíciles de rastrear. `isna()` devuelve un DataFrame booleano; `.sum()` cuenta los True por columna; `.sum().sum()` suma todo el DataFrame.

¿Por qué mediana para usuarios_afectados y no media? el incidente 1 tiene 12.000 usuarios afectados (un valor extremo comparado con los demás). La media sin ese valor es ~5.540 y con él es ~6.633. La mediana (6.700) es más estable ante ese outlier.

Interpretación de negocio: el 15% de celdas con nulos es una señal de alerta. En un proyecto real, antes de imputar se investigaría la causa: ¿falla del sistema de registro? ¿incidentes durante mantenimiento donde el registro no aplica? La estrategia de imputación debe estar documentada y justificada para el equipo de negocio.

Paso 5: Columnas derivadas y groupby

groupby segmenta el DataFrame según los valores únicos de una columna y aplica funciones de agregación a cada segmento. Es el equivalente Pandas del GROUP BY de SQL, pero más flexible: puedes aplicar múltiples funciones simultáneamente con agg().

```
# Columna derivada: ingresos estimados
df_consumo['ingresos_cop'] = df_consumo['consumo_kwh'] *
df_consumo['tarifa_kwh']
print('Primeras 4 filas con columna de ingresos:')
print(df_consumo[['municipio', 'mes', 'consumo_kwh', 'tarifa_kwh', 'ingresos_cop']].head(4))

# Verificación manual Medellín enero:
# 145.230.000 kWh × 650 COP/kWh = 94.399.500.000 COP
print(f'\nVerificación: 145230000 × 650 = {145230000 * 650:,}')

```

Primeras 4 filas con columna de ingresos:

	municipio	mes	consumo_kwh	tarifa_kwh	ingresos_cop
0	Medellín	2024-01	145230000	650	94399500000
1	Bello	2024-01	28450000	640	18208000000
2	Envigado	2024-01	19870000	655	13014850000
3	Itagüí	2024-01	22100000	645	14254500000

Verificación: 145230000 × 650 = 94,399,500,000

```
# groupby: resumen por municipio (sobre los 2 meses combinados)
resumen = df_consumo.groupby('municipio').agg(
    consumo_total = ('consumo_kwh', 'sum'),
    consumo_prom = ('consumo_kwh', 'mean'),
    ingresos_total = ('ingresos_cop', 'sum'),
    meses_registrados = ('mes', 'count')
).round(0).reset_index()

print(resumen)

```

	municipio	consumo_total	consumo_prom	ingresos_total
0	Bello	56340000	28170000.0	36566500000
2				
1	Envigado	39210000	19605000.0	25876450000
2				
2	Itagüí	43880000	21940000.0	28554000000
2				
3	Medellín	287120000	143560000.0	188047400000
2				

Interpretación: Paso 5

Interpretación técnica: `agg()` con sintaxis de diccionario (`nombre_columna_resultado = ('columna_fuente', 'funcion')`) permite crear múltiples métricas con nombres descriptivos en una sola llamada. `reset_index()` convierte el índice del `groupby` de vuelta en columna normal.

Interpretación de negocio: Medellín genera 188.000 millones COP en ingresos acumulados de dos meses, más del 75% del total del área metropolitana. Los ingresos de Medellín son 5.1 veces los de Bello, aunque Medellín tiene solo 5 veces más usuarios. La diferencia se explica por la tarifa ligeramente mayor (655 vs 645 promedio).

Paso 6: Combinar tablas — merge

`merge()` es el JOIN de Pandas. Une dos DataFrames usando una columna clave en común. Los parámetros fundamentales son: `on` (columna de unión), `how` ('inner', 'left', 'right', 'outer'). El comportamiento es idéntico al JOIN de SQL.

```
# DataFrame de presupuesto de mantenimiento (simula datos de un archivo Excel)
df_presupuesto = pd.DataFrame({
    'municipio':          ['Medellín', 'Bello', 'Envigado',
                           'Itagüí', 'Sabaneta'],
    'presupuesto_mant':    [850000000, 210000000, 150000000,
                           175000000, 95000000],
    'tecnicos_asignados':  [42,          18,          12,          14,
                           8]
})

# INNER JOIN: solo municipios presentes en AMBOS DataFrames
# (Sabaneta queda fuera porque no tiene datos de consumo)
df_inner = df_consumo[df_consumo['mes']=='2024-03'].merge(
    df_presupuesto,
    on='municipio',
    how='inner'
)

print('INNER JOIN (4 filas - Sabaneta excluida):')
print(df_inner[['municipio', 'consumo_kwh', 'presupuesto_mant', 'tecnicos_asignados']])
```

```
INNER JOIN (4 filas - Sabaneta excluida):
  municipio  consumo_kwh  presupuesto_mant  tecnicos_asignados
0  Medellín    141890000         850000000             42
1    Bello     27890000         210000000             18
```

2	Envigado	19340000	150000000	12
3	Itagüí	21780000	175000000	14

```
# LEFT JOIN: todos los municipios del presupuesto, con o sin datos de
consumo
# (Sabaneta aparece con NaN en las columnas de consumo)
df_left = df_presupuesto.merge(
    df_consumo[df_consumo['mes']=='2024-03'],
    on='municipio',
    how='left'
)
print('\nLEFT JOIN (5 filas - Sabaneta con NaN):')
print(df_left[['municipio', 'presupuesto_mant', 'consumo_kwh', 'num_usuarios'
]])
```

```
LEFT JOIN (5 filas - Sabaneta con NaN):
```

	municipio	presupuesto_mant	consumo_kwh	num_usuarios
0	Medellín	850000000	141890000.0	898000.0
1	Bello	210000000	27890000.0	180000.0
2	Envigado	150000000	19340000.0	126000.0
3	Itagüí	175000000	21780000.0	140000.0
4	Sabaneta	95000000	NaN	NaN

```
# Columna derivada post-merge: costo de mantenimiento por usuario
df_inner['costo_por_usuario'] = (
    df_inner['presupuesto_mant'] / df_inner['num_usuarios']
).round(2)
print('\nCosto de mantenimiento por usuario (COP):')
print(df_inner[['municipio', 'num_usuarios', 'presupuesto_mant', 'costo_por_u
suario']])
```

```
Costo de mantenimiento por usuario (COP):
```

	municipio	num_usuarios	presupuesto_mant	costo_por_usuario
0	Medellín	898000	850000000	946.55
1	Bello	180000	210000000	1166.67
2	Envigado	126000	150000000	1190.48
3	Itagüí	140000	175000000	1250.00

Interpretación: Paso 6

INNER vs LEFT: INNER JOIN devuelve solo la intersección (filas con clave en ambos DataFrames). LEFT JOIN devuelve todo el DataFrame izquierdo, con NaN donde no hay coincidencia. Elegir el tipo correcto es una decisión de negocio: si Sabaneta tiene presupuesto asignado, no debería desaparecer del reporte aunque no tenga datos de

consumo este mes.

Interpretación de negocio: Medellín tiene el menor costo por usuario (946.55 COP) a pesar de tener el mayor presupuesto absoluto (850M), gracias a sus economías de escala con 898.000 usuarios. Itagüí tiene el mayor costo por usuario (1.250 COP), lo que puede justificar una revisión de su asignación presupuestal.

Paso 7: Apilar DataFrames — concat

`pd.concat()` apila DataFrames verticalmente (por defecto) o los combina horizontalmente. Es distinto de `merge`: `concat` no une por columna clave, simplemente suma filas de DataFrames con la misma estructura. Útil para combinar datos del mismo tipo de múltiples períodos o múltiples archivos.

```
# Simular dos meses ya cargados como DataFrames separados
df_enero = df_consumo[df_consumo['mes'] == '2024-01'].copy()
df_marzo = df_consumo[df_consumo['mes'] == '2024-03'].copy()

print(f'Filas enero: {len(df_enero)}, filas marzo: {len(df_marzo)}')

# concat: apilar verticalmente, reiniciar el índice
df_todos = pd.concat([df_enero, df_marzo], ignore_index=True)

print(f'Filas combinadas: {len(df_todos)}')
print(df_todos)
```

```
Filas enero: 4, filas marzo: 4
Filas combinadas: 8
  municipio    mes  consumo_kwh  num_usuarios  tarifa_kwh  ingresos_cop
0  Medellín  2024-01    145230000         892000         650    94399500000
1    Bello    2024-01    28450000         178000         640    18208000000
2  Envigado  2024-01    19870000         124000         655    13014850000
3   Itagüí    2024-01    22100000         138000         645    14254500000
4  Medellín  2024-03    141890000         898000         660    93647400000
5    Bello    2024-03    27890000         180000         650    18128500000
6  Envigado  2024-03    19340000         126000         665    12861100000
7   Itagüí    2024-03    21780000         140000         655    14265900000
```

```
# groupby sobre el DataFrame combinado: evolución por municipio
evolucion = df_todos.groupby('municipio').agg(
    consumo_total = ('consumo_kwh', 'sum'),
    ingreso_total = ('ingresos_cop', 'sum')
```

```
.reset_index()  
print(evolucion)
```

```
  municipio  consumo_total  ingreso_total  
0      Bello      56340000      36336500000  
1  Envigado      39210000      25875950000  
2    Itagüí      43880000      28520400000  
3  Medellín      287120000      188046900000
```

Interpretación: Paso 7

Interpretación técnica: ignore_index=True reinicia el índice del DataFrame resultante (0, 1, 2... en lugar de conservar los índices originales de cada fragmento). Si no se usa, pueden quedar índices duplicados que causan comportamientos inesperados en operaciones posteriores.

merge vs concat: merge une DataFrames por columna clave (como un JOIN: une información de diferentes 'tipos' de datos). concat apila DataFrames con la misma estructura (como agregar más filas del mismo 'tipo' de datos). Una analogía: merge es como unir dos hojas de Excel por una columna común; concat es como pegar una hoja debajo de la otra.

Interpretación de negocio: el consumo de Medellín cayó de 145.2M a 141.9M kWh entre enero y marzo (-2.3%). Bello tuvo la mayor caída proporcional (28.5M a 27.9M kWh, -2.1%). Estos análisis de tendencia requieren exactamente esta estructura: concat para combinar períodos, groupby para resumir.

SECCIÓN 3 | Cheat Sheet Pandas

Funciones de ingesta

Función	Sintaxis básica	Para qué sirve
pd.read_csv()	pd.read_csv('archivo.csv')	Lee CSV. Parámetros útiles: sep, encoding, parse_dates, usecols, dtype.
pd.read_excel()	pd.read_excel('archivo.xlsx', sheet_name=0)	Lee Excel. sheet_name puede ser nombre o índice de hoja.
pd.read_json()	pd.read_json('archivo.json') o pd.DataFrame(lista)	Lee JSON. Funciona con archivos o con listas de diccionarios de Python.

<code>pd.read_sql()</code>	<code>pd.read_sql(query, conexion)</code>	Ejecuta SQL y devuelve el resultado como DataFrame. Requiere conexión activa.
----------------------------	---	---

Exploración inicial

Atributo / Método	Qué devuelve
<code>df.shape</code>	Tupla (filas, columnas).
<code>df.dtypes</code>	Tipo de dato de cada columna.
<code>df.head(n)</code>	Primeras n filas (defecto 5).
<code>df.tail(n)</code>	Últimas n filas.
<code>df.info()</code>	Resumen: tipos, nulos, memoria.
<code>df.describe()</code>	Estadísticas descriptivas de columnas numéricas.

Calidad de datos

Método	Uso y ejemplo
<code>df.isna()</code>	Devuelve DataFrame booleano: True donde hay NaN.
<code>df.isna().sum()</code>	Cuenta nulos por columna.
<code>df['col'].fillna(valor)</code>	Rellena NaN con un valor fijo o con una estadística: <code>fillna(df['col'].mean())</code> .
<code>df.dropna()</code>	Elimina filas con al menos un NaN. Usar con cuidado: puede perder datos valiosos.
<code>df.copy()</code>	Crea una copia independiente del DataFrame. SIEMPRE usar antes de modificar.

Transformación y análisis

Operación	Código	Nota
Columna derivada	<code>df['nueva'] = df['a'] * df['b']</code>	Operación vectorizada, sin loops.
groupby simple	<code>df.groupby('col').agg({'val': 'sum'})</code>	Agrupar y agregar en un paso.
groupby múltiple	<code>df.groupby('col').agg(total=('val', 'sum'), prom=('val', 'mean'))</code>	Múltiples métricas con nombres descriptivos.
<code>reset_index()</code>	<code>df.groupby(...).agg(...).reset_index()</code>	Convierte el índice de groupby en columna normal.
merge inner	<code>df1.merge(df2, on='clave', how='inner')</code>	Solo filas con coincidencia en

		ambos DataFrames.
merge left	df1.merge(df2, on='clave', how='left')	Todas las filas de df1, NaN donde no hay coincidencia.
concat	pd.concat([df1, df2], ignore_index=True)	Apila DataFrames verticalmente (más filas).

SECCIÓN 4 | Ejercicio Propuesto Autónomo

Contexto: Postobón — Análisis de ventas por canal y producto

Postobón es la empresa de bebidas no alcohólicas más grande de Colombia y una de las más importantes de América Latina. Fue fundada en 1904 en Medellín por Gabriel Posada y Valerio Tobón (de ahí su nombre: POS-TO-BÓN). Hace parte del Grupo Ardila Lülle y comercializa marcas como Postobón, Colombiana, Manzana Postobón, Mr. Tea, Hit y agua Cristal, entre otras. Tiene plantas de producción en más de 17 ciudades colombianas y distribuye en todo el territorio nacional.

Caso de uso: el equipo de analítica de Postobón quiere integrar datos de ventas de tres canales (supermercados en CSV, tiendas de barrio en Excel, y datos históricos en una base SQL) para generar un reporte consolidado de desempeño por producto y región.

Crea los siguientes conjuntos de datos:

Fuente	Estructura sugerida
CSV: ventas_supermercados	producto, region, mes, unidades_vendidas, precio_unitario (al menos 10 filas, 3 productos, 2 regiones)
Excel (simular con DataFrame)	producto, region, mes, unidades_vendidas, precio_unitario para el canal tiendas_barrio (misma estructura)
SQL: tabla ventas_historicas	id, producto, region, año, unidades_anuales (datos de 2022 y 2023)
DataFrame con nulos: calidad_control	id_lote, producto, planta, temperatura_proceso, ph_medido, resultado_inspeccion (introducir al menos 4 nulos en distintas columnas)

Puntos del ejercicio

Punto 1 — Ingesta múltiple

- Carga el CSV con `read_csv` y el Excel (simulado con `pd.DataFrame`) con las mismas columnas. Verifica que ambos tienen la misma estructura con `.dtypes`.

- Carga los datos de ventas_historicas desde SQLite con `pd.read_sql` usando una consulta que agrupe por producto y calcule el total de unidades por año.
- En tu celda Markdown: ¿qué diferencias encuentras entre los formatos de ingesta? ¿Cuándo usarías `read_sql` en lugar de cargar el archivo completo y filtrar en Pandas?

Punto 2 — Calidad de datos

- Aplica `isna().sum()` al DataFrame de calidad_control. Muestra el porcentaje de nulos por columna.
- Define una estrategia de imputación para cada columna con nulos. Justifica tu elección (media, mediana, valor constante, dropna). Aplica `fillna()` con tu estrategia.
- Verifica que después del `fillna()` no quedan nulos.

Punto 3 — Columnas derivadas y groupby

- Crea la columna `ingresos_cop = unidades_vendidas * precio_unitario` en ambos DataFrames (supermercados y tiendas).
- Aplica `groupby` por producto para calcular: total de unidades vendidas, total de ingresos, precio promedio.
- ¿Qué producto tiene el mayor ingreso total? ¿Y el mayor volumen de unidades?

Punto 4 — merge y concat

- Usa `pd.concat` para apilar los DataFrames de supermercados y tiendas en un único DataFrame. Agrega una columna 'canal' que identifique el origen de cada fila antes de concatenar.
- Usa `merge (inner)` para unir el DataFrame consolidado con un pequeño DataFrame de metas mensuales por producto (créalo tú: producto, meta_unidades).
- ¿Qué productos superan su meta? Filtra el DataFrame resultante.

Punto 5 — Análisis integrado

- A partir del DataFrame consolidado (concat de los dos canales), calcula con `groupby` el ingreso total por región y por canal.
- ¿Qué canal genera más ingresos en cada región?
- En tu celda Markdown: ¿en qué fase de CRISP-DM está este ejercicio completo? ¿Qué haría falta para llegar a la Fase 4 (Modelado)?

Criterio de autoevaluación

Tu trabajo está completo si: (1) los cuatro tipos de ingesta (CSV, Excel, JSON/DataFrame, SQL) están implementados y puedes explicar cuándo usar cada uno; (2) cada decisión de imputación está justificada por las características de la variable, no elegida al azar; (3) `concat` y `merge` están usados correctamente y puedes explicar la diferencia en una frase.

Pista: la columna 'canal' que agregas antes del concat es un ejemplo de 'ingeniería de features' (feature engineering): crear nueva información que no existía en los datos originales pero que aporta valor al análisis. Este concepto es central en la Fase 3 de CRISP-DM y en todo el módulo de Machine Learning.